

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Srushti N (1BM24CS424)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “Machine Learning (23CS6PCMAL)” carried out by **Srushti N (1BM24CS424)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Megha J Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

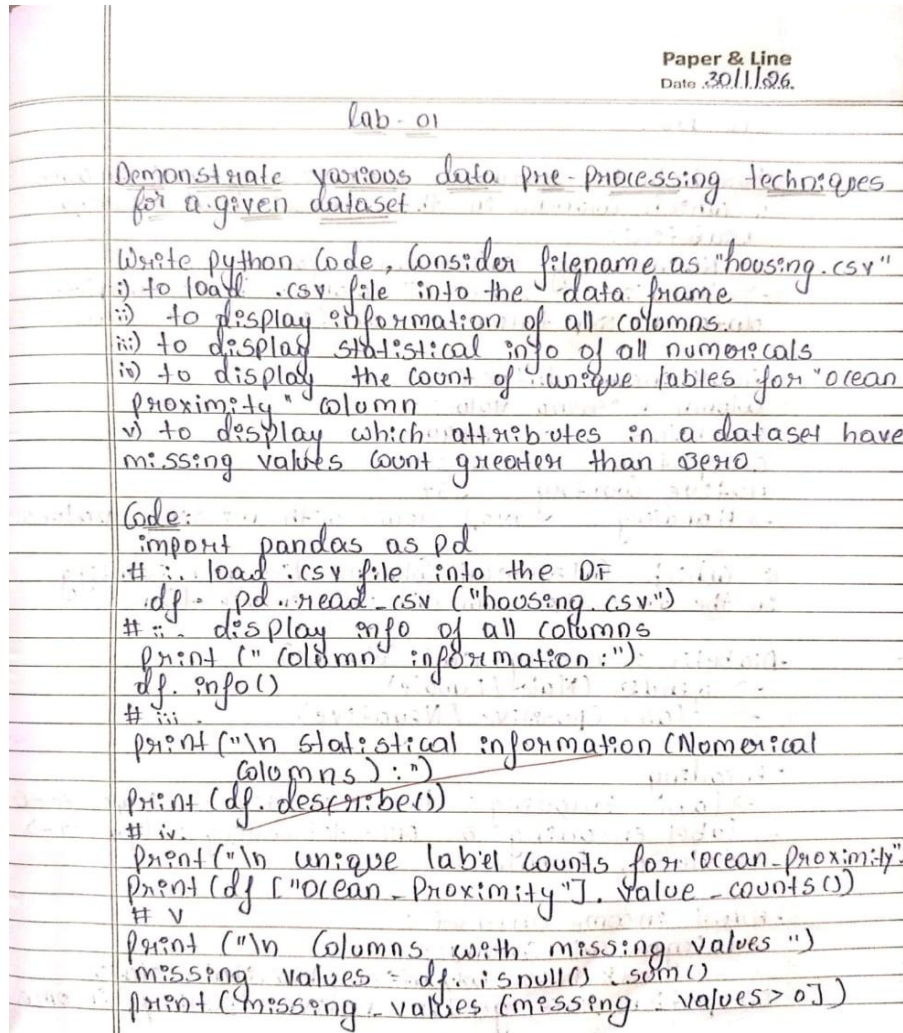
Sl. No.	Date	Experiment Title	Page No.
1	30-01-26	Write a python program to import and export data using Pandas library functions	4-7
2	06-02-26	Demonstrate various data pre-processing techniques for a given dataset	8-12
3	13-02-26	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	13-14
4	27-02-26	Build Logistic Regression Model for a given dataset	15-17
5	06-03-26	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	18-19
6	13-03-26	Build KNN Classification model for a given dataset.	20-22
7	27-03-26	Build Support vector machine model for a given dataset	23-24
8	27-03-26	Implement Random forest ensemble method on a given dataset.	25-26
9	03-04-26	Implement Boosting ensemble method on a given dataset.	27-28
10	10-04-26	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	29-30
11	10-04-26	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	31-32

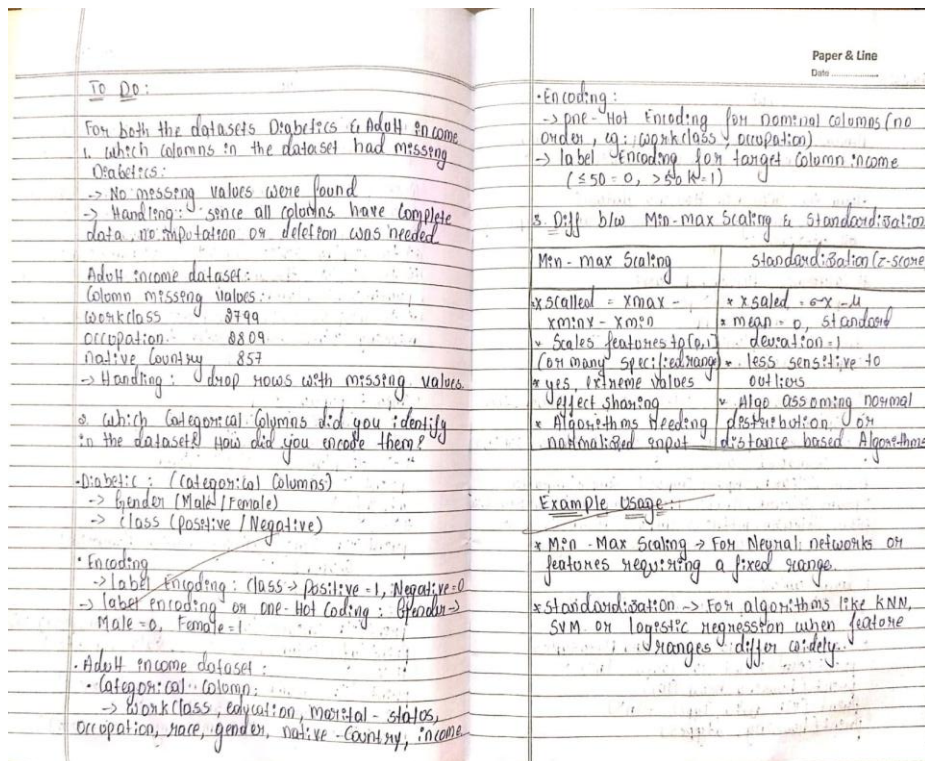
Github Link: https://github.com/srushtinagaraju/Machine_Learning_1BM24CS424.g

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot





Code:

```
df['species'].value_counts()

...
count
species
0      50
1      50
2      50

dtype: int64

# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```

```
df['species'].value_counts()
```

```
...          count
species
0           50
1           50
2           50
```

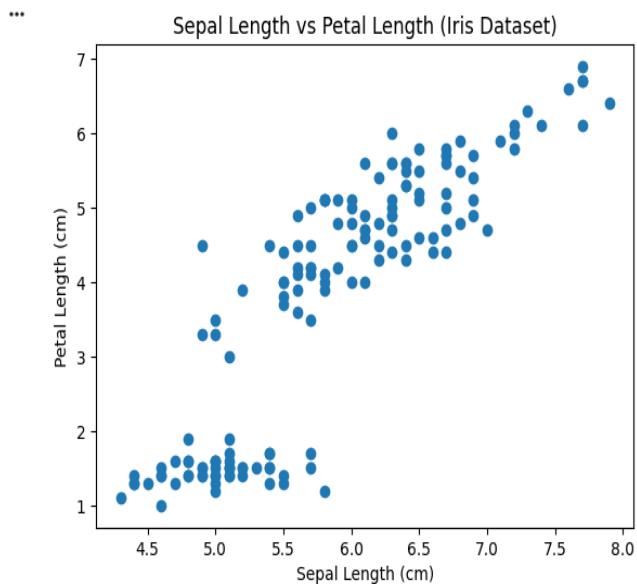
dtype: int64

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```



df.corr()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

df.sort_values(by='petal length (cm)').head()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

df[df['species'] == 0].head()

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

df[df['species'] == 1].head()

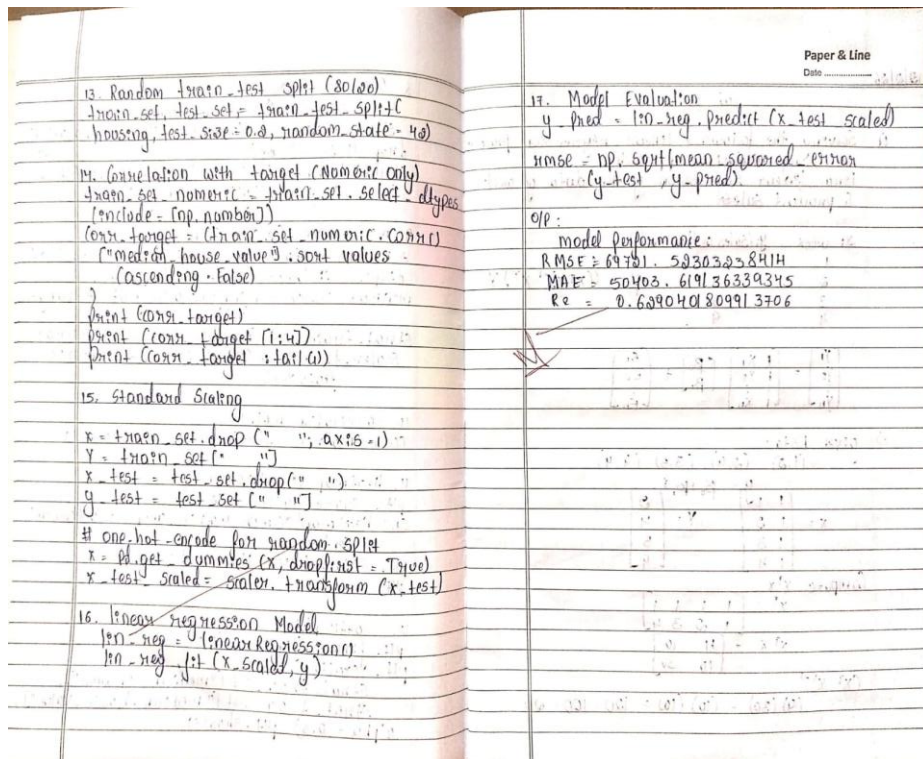
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot

```
06/06/2020 Lab - 02  
Demonstrate various data pre-processing techniques for a given dataset  
• Load the California Housing dataset  
• Display the first 10 rows & data types of all attributes  
1. California Housing Dataset  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import MinMaxScaler, StandardScaler  
from sklearn.impute import SimpleImputer  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error, r2_score  
1. Load dataset  
url = "housing.csv" # read csv(url) "  
2. Display first 10 rows and Data types  
print("\n First 10 rows : \n")  
print(housing.head(10))  
print("\n Data Types : \n")  
print(housing.dtypes)  
3. Basic Statistics  
print("\n Basic Statistics : \n")  
print(housing.describe())  
or :  
longitude latitude housing median age total rooms  
149.0 37.8 41.0 880.0  
total bedrooms population households median income  
149.0 300.0 106.0 8.3050  
median house values ocean proximity  
452600.0 NEAR BAY  
Data types :  
longitude float64  
latitude float64  
ocean proximity object  
4. Target Variable Distribution  
plt.figure(figsize=(8,5))  
plt.hist(housing['median house value'], bins=50)  
plt.xlabel("Median House Value")  
plt.ylabel("Frequency")  
plt.title("Histogram of Median House Value")  
plt.show()  
plt.figure(figsize=(6,4))  
plt.boxplot(housing[" "])  
plt.xlabel("Median House Value")  
plt.title("Box Plot of Median House Value")  
plt.show()  
5. Missing values Analysis  
print("\n Missing values (%) : \n")  
print(housing.isnull().mean()*100)  
or :  
Missing values (%) :  
longitude 0.00000  
latitude 0.00000  
housing median age 0.00000  
total rooms 0.00000  
total bedrooms 1.003907  
population 0.00000  
households 0.00000  
median income 0.00000  
median house value 0.00000  
ocean proximity 0.00000  
6. Handle missing values (Median Imputation)  
imputer = SimpleImputer(strategy="median")  
housing["total bedrooms"] = imputer.fit_transform(housing[["total bedrooms"]])  
7. Stratified sampling on median income  
(Before Stratifying housing["income cat"] = pd.cut(housing["median income"], bins=(0, 1.5, 3.0, 4.5, 6, np.inf), labels=[1, 2, 3, 4, 5])  
strat_train_set, strat_test_set = train_test_split(housing, test_size=0.2, stratify=housing["income cat"], random_state=42)  
for set in (strat_train_set, strat_test_set):  
set.drop("income cat", axis=1, inplace=True)  
8. One-Hot Coding  
strat_train_set = pd.get_dummies(strat_train_set, columns=["ocean proximity"])  
drop_first = True  
strat_train_set, strat_test_set, strat_train_set, strat_test_set, join="left", axis=1, fill_value=0)  
9. min-max Scaling (0-1)  
-> numeric features: 0 = strat_train_set.select_dtypes(include=[np.number]).columns  
minmax_scaler = MinMaxScaler(feature_range=(0,1))  
strat_train_set[numerical_features] = minmax_scaler.fit_transform(strat_train_set[numerical_features])  
10. Correlation matrix  
corr_matrix = strat_train_set.corr()  
11. Heatmap visualization  
plt.figure(figsize=(10,8))  
sns.heatmap(corr_matrix, cmap="coolwarm", center=0)  
plt.title("Correlation Matrix heatmap")  
plt.show()  
12. Scatter Plot  
plt.figure(figsize=(7,5))  
plt.scatter(strat_train_set["median income"], strat_train_set["median house value"], alpha=0.3)  
plt.show()
```

Code:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

...	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

df.shape

(150, 5)

df.columns

Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
 'petal width (cm)', 'species'],
 dtype='object')

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   sepal length (cm)    150 non-null    float64
 1   sepal width (cm)     150 non-null    float64
 2   petal length (cm)    150 non-null    float64
 3   petal width (cm)     150 non-null    float64
 4   species              150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
df.describe()
```

```
***
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```
df['species'].value_counts()
```

```
***
```

	count
species	
0	50
1	50
2	50

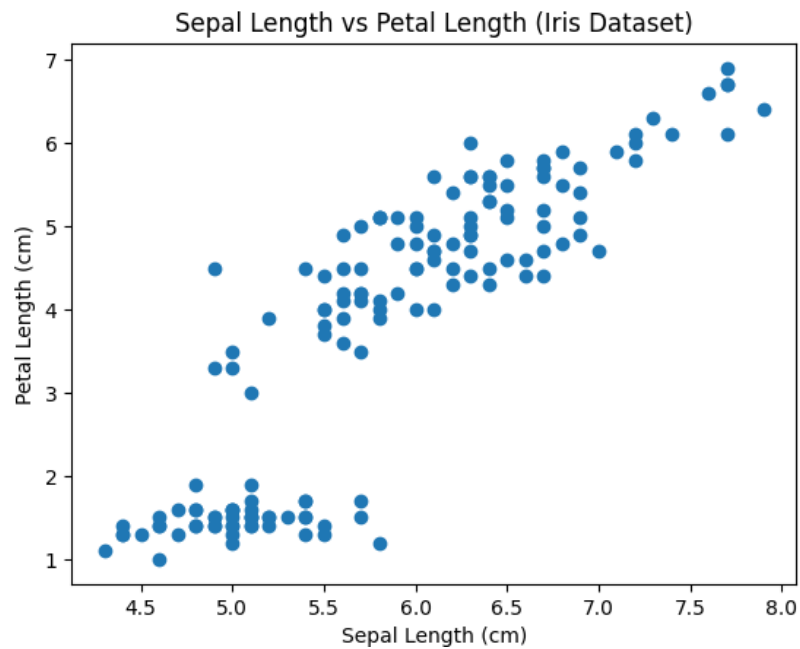
```
dtype: int64
```

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```



```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0



```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

df.corr()

***	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

df.sort_values(by='petal length (cm)').head()

***	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot

18/10/26

Lab-03

A Solving the following linear regression problem using matrix approach.

Find linear regression of the data of week & product sales

Week	Sales (in thousands)
1	0
2	4
3	5
4	9

$$\beta = ((X^T X)^{-1} X^T) Y$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ 1 & x_4 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \\ \epsilon_4 \end{bmatrix}$$

Given Data: (1,0) (2,4) (3,5) (4,9)

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \quad Y = \begin{bmatrix} 0 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$

Compute $X^T X$

$$X^T X = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

$$(X^T X)^{-1} = \frac{1}{(4)(30) - (10)(10)} = \frac{1}{120 - 100} = \frac{1}{20}$$

Paper & Line
Date

$$(X^T X)^{-1} = \frac{1}{20} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix} = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix}$$

$$X^T Y = \begin{bmatrix} 0+4+5+9 \\ (1)(0) + (2)(4) + (3)(5) + (4)(9) \end{bmatrix} = \begin{bmatrix} 18 \\ 61 \end{bmatrix}$$

Compute β

$$\beta = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix} \begin{bmatrix} 18 \\ 61 \end{bmatrix}$$

$$\beta_0 = 1.5(18) - 0.5(61) = 27 - 30.5 = -3.5$$

$$\beta_1 = -0.5(18) + 0.2(61) = -9 + 12.2 = 3.2$$

$$y = -3.5 + 3.2x$$

Write the answers for the following question

1. Considering the three data.csv files (Canada per capita income.csv, Salary.csv, hiring.csv) Did you perform any data preprocessing steps (e.g., missing values, scaling, encoding)?

Yes

- hiring.csv - converted experience from words to numbers, filled missing values
- Salary.csv - no major preprocessing needed
- Canada per capita income.csv - no missing values, no preprocessing needed

2. For Canada per capita income.csv, did you visualize the regression line along with the data points? What does the plot tell you about the relationship b/w year & per capita income

⇒ The plot of per capita income versus year shows a strong positive linear relationship, meaning that per capita income in Canada has been increasing steadily over the years.

3. hiring.csv

⇒ The predicted salary for a candidate with 10 years of experience, 40 test score, & 10 interview score is \$92,268.07

4. 1000 Companies.csv

⇒ The Categorical variable state was encoded using one-hot encoding, which creates separate binary columns for each state. No scaling of feature was applied b/c linear regression can handle unscaled numerical data. Using the trained model, the predicted profit for the given company data is \$51,071.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(criterion='entropy', random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("Predicted class:", prediction[0])
print("Predicted species:", iris.target_names[prediction[0]])
```

```
*** Accuracy: 1.0
    Predicted class: 0
    Predicted species: setosa
```


Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot

```
Paper & Line
Date: 21/10/2020

Lab - 04
Logistic Regression

Logistic Regression for Binary classification.
#1
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix

#2
df = pd.read_csv("HR_comma_sep.csv")
print("\n D = ", df.head())
print("\n = ", df.shape)
print("\n Column: ", df.columns)

#3
print("\n = ", df["left"].value_counts())
sns.countplot(x="left", data=df)
plt.title("Employee Retention (0 = Stayed, 1 = Left)")
plt.show

#4
salary_retention = pd.melt(df, id_vars="salary", df
"left")
salary_retention.plot(kind="bar")
```

```
accuracy = accuracy_score(y_test, y_pred)
plt.figure(figsize=(8,6))
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Zoo Classification")
plt.show()

Question and Answer

1. HR comma sep.csv
i) variables affecting employee retention:
satisfaction level, number of projects, avg
monthly hours.
ii) Accuracy of logistic regression model:
Accuracy was around 80%.

Zoo datasets
i) Data processing steps: yes
ii) Missing or inconsistent values: No
iii) Confusion matrix meaning: it shows
the number of correct values
iv) Misclassified class types:
Animals with similar features
were some times misclassification
```

```
Paper & Line
Date:

Salary vs Employee Retention
plt.title("Salary vs Employee Retention")
plt.xlabel("Salary Level")
plt.ylabel("Number of Employees")
plt.xticks(rotation=45)
plt.show()

#5
if 'Dept' in df.columns:
    dept_col = "Dept"
else:
    dept_col = "Sales"

dept_retention = pd.melt(df, id_vars=dept_col, df
"left")
dept_retention.plot(kind="bar", figsize=(10,8))
plt.title(" ")
plt.xlabel("Dept")
plt.ylabel("No. of Employees")
plt.xticks(rotation=45)
plt.show()

#6
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\n Model accuracy: ", round
(accuracy*100,2), "%")

Multiclass Classification - Logistic Regression

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

from google.colab import files
from sklearn.model_selection import train
test_split

from sklearn.linear_model import LogisticRegression

# Load Dataset
zoo_df = pd.read_csv("zoo file")
class_df = pd.read_csv("class file")
print(zoo_df.head())
print(class_df)

# Data Processing
if 'animal_name' in zoo_df.columns:
    zoo_df = zoo_df.drop("animal_name")
print(zoo_df.head())
print(class_df)

if 'animal_name' in zoo_df.columns:
    zoo_df = zoo_df.drop("animal_name")
x = zoo_df.drop("class", axis=1)
y = zoo_df["class"]
```

Code:

```
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
▶ scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
▶ lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

y_pred_lr = lin_reg.predict(X_test_scaled)

rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

rmse_lr, mae_lr, r2_lr

... (np.float64(0.7449540928043491), 0.5330139776645934, 0.5765012224420747)
```

```
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

rmse_dt

np.float64(0.7138377373947808)
```

```
▶ rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

rmse_rf

... np.float64(0.5128644706054181)
```

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    rf,
    param_grid,
    cv=3,
    scoring='neg_mean_squared_error'
)
```

```
results = pd.DataFrame({
    'Model': ['Linear Regression',
             'Decision Tree',
             'Random Forest',
             'Tuned Random Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})

results
```

```
***
```

	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Program 5

Build Logistic Regression Model for a given dataset

Screenshot

Paper & Line
Date: 6/03/26

Lab-05

KNN classification model solve

$k=3$

Test data: $X = (\text{Age} = 35, \text{Salary} = 100)$

Dataset:

Person	Age	Salary	Target
A	18	50	N
B	23	55	N
C	24	70	N
D	41	60	Y
E	43	70	Y
F	38	40	Y

Calculating euclidean distance

$d = \sqrt{(\text{age}_i - \text{age}_x)^2 + (\text{salary}_i - \text{salary}_x)^2}$

distance from $x (35, 100)$

A (18, 50) $d = \sqrt{(35-18)^2 + (100-50)^2}$
 $= \sqrt{2789}$
 $= 52.82$

B (23, 55) $d = \sqrt{(35-23)^2 + (100-55)^2}$
 $= \sqrt{2169}$
 $= 46.57$

C (24, 70) $d = \sqrt{(35-24)^2 + (100-70)^2}$
 $= \sqrt{1021}$
 $= 31.94$

by standardization:

$X_{\text{scaled}} = \frac{X - \text{mean}}{\text{std. Deviation}}$

Results:

For this dataset: accuracy: 1.0

Confusion matrix: $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

For diabetes dataset: accuracy: 0.694444

Confusion matrix: $\begin{bmatrix} 79 & 20 \\ 24 & 28 \end{bmatrix}$

For heart dataset: accuracy: 0.9180327

Confusion matrix: $\begin{bmatrix} 27 & 2 \\ 3 & 29 \end{bmatrix}$

Paper & Line
Date: _____

D (41, 60) $d = \sqrt{1634}$
 $= 40.45$

E (43, 70) $d = \sqrt{964}$
 $= 31.05$

F (38, 40) $d = \sqrt{3609}$
 $= 60.07$

sort distance

Person	Distance	Target
E	31.05	Y
C	31.94	N
D	40.45	Y
B	46.57	N
A	52.82	N
F	60.07	Y

Select $k=3$ nearest neighbours

E Y
C N
D Y

Y = 2
N = 1

majority class = Y

Target for $x (35, 100) = Y$

For this dataset

How to choose the k value? Demo with accurate rate & error rate

→ The value of k represents the no. of nearest neighbours used to classify a new data point.

small k → overfit the data
 large k → underfit the data

To choose the best k , we calculate accuracy rate and error rate for different k values, and select the k with highest accuracy and lowest error.

Steps:

1. Train KNN model for multiple k value
2. Calculate accuracy rate for each k
3. Compute error rate = $1 - \text{accuracy}$
4. select k

What is the purpose of feature scaling?
 How to perform it?

→ Feature scaling is used to normalise the range of independent variables so that all features contribute equally to the model.

This is needed because if calculate distance between data points, if one feature has a large range, then others it will dominate the distance calculation. It ensures all features are on similar scale. Feature scaling can be performed

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt

# Load correct file
iris = pd.read_csv("iris (1).csv")

print(iris.head())

# Features and target
X = iris.drop('species', axis='columns')
y = iris['species']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
model.fit(X_train, y_train)

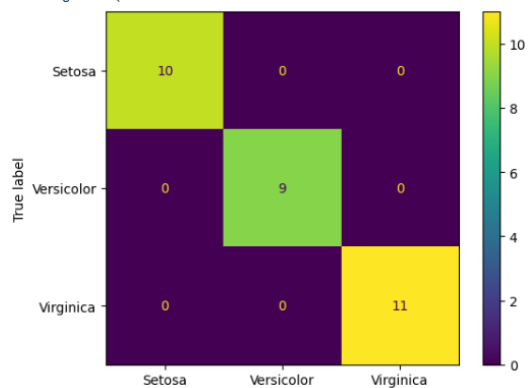
# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = metrics.confusion_matrix(y_test, y_pred)
disp = metrics.ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Setosa", "Versicolor", "Virginica"]
)
disp.plot()
plt.show()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Accuracy: 1.0
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated i
warnings.warn()



Program 6

Build KNN Classification model for a given dataset

Screenshot

Paper & Line
Date: 15/3/2021

Lab - 06

Consider the following dataset, calculate Entropy and information gain w.r.t to target variable "Classification". Identify whether the splitting node should be a_2 or a_3 attribute.

Instance	a_2	a_3	Classification
1	Hot	High	No
2	Hot	High	No
6	Cool	High	No
7	Hot	High	No
8	Hot	Normal	Yes

$\text{YES} = 1$
 $\text{NO} = 0$
 Total = 5
 Entropy formula: $H(S) = -p_{\text{yes}} \log_2(p_{\text{yes}}) - p_{\text{no}} \log_2(p_{\text{no}})$
 $p_{\text{yes}} = \frac{1}{5} = 0.2, p_{\text{no}} = \frac{4}{5} = 0.8$
 $H(S) = -(0.2 \log_2 0.2 + 0.8 \log_2 0.8)$
 $H(S) = -(0.2(-2.3219)) + 0.8(-0.3219)$
 $H(S) = 0.7219$
 Entropy (S) = 0.7219

Paper & Line
Date: 15/3/2021

Information gain for attribute a_2

Split by a_2

Hot: (1, 2, 7, 8)

- YES = 1
- NO = 3
- $H(\text{Hot}) = 0.8113$

Cool: (6)

- YES = 0
- NO = 1
- $H(\text{Cool}) = 0$

Weighted Entropy:

$$H(S|a_2) = \frac{4}{5} (0.8113) + \frac{1}{5} (0)$$

$$H(S|a_2) = 0.649$$

Information gain

$$I_{a_2}(S, a_2) = H(S) - H(S|a_2)$$

$$I_{a_2}(S, a_2) = 0.7219 - 0.649$$

$$I_{a_2}(S, a_2) = 0.0729$$

Information gain for attribute a_3

Split by a_3

High: (1, 2, 6, 7)

- YES = 0
- NO = 4
- Entropy = 0

Normal: (8)

- YES = 1
- NO = 0
- Entropy = 0

Weighted Entropy:

$$H(S|a_3) = \frac{4}{5} (0) + \frac{1}{5} (0) = 0$$

Information gain

$$I_{a_3}(S, a_3) = 0.7219 - 0$$

$$I_{a_3}(S, a_3) = 0.7219$$

Splitting attribute = a_3
 $a_3 (0.7219) > a_2 (0.0729)$

Q > For this CSV dataset

Accuracy score:
 The decision tree model achieved high accuracy on the IIS dataset.

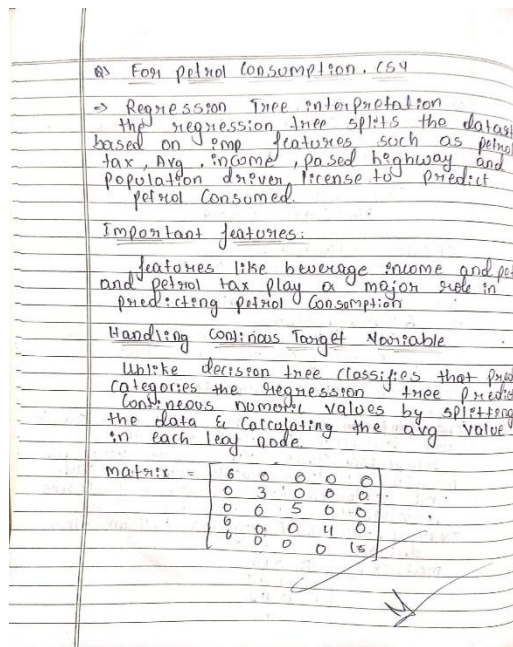
Confusion matrix interpretation

The Confusion matrix shows that most sample are correctly classified into their respective class. There are very few misclassifications, mainly b/w Versicolour and Virginica because their features are similar.

Overall, the model performs well on this dataset.

Matrix =

16	0	0
0	9	0
0	0	11



Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```

regressor = DecisionTreeRegressor()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)

```

```

Mean Squared Error: 5358.458646616541
R2 Score: 0.0073817790735694055

```

```

from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()

X = data.data
y = data.target

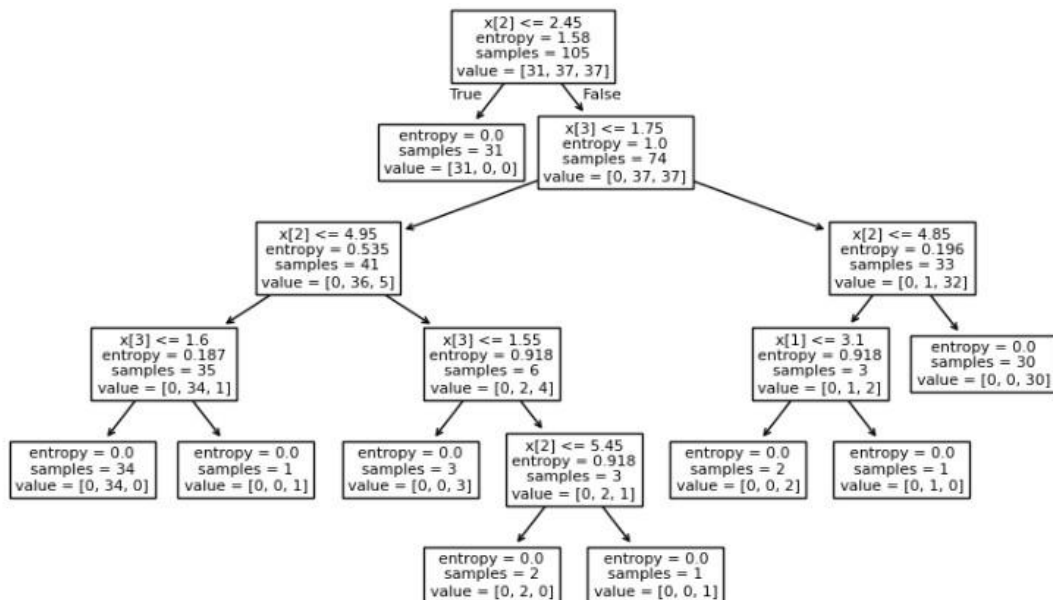
```

```

▶ from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(model)
plt.show()

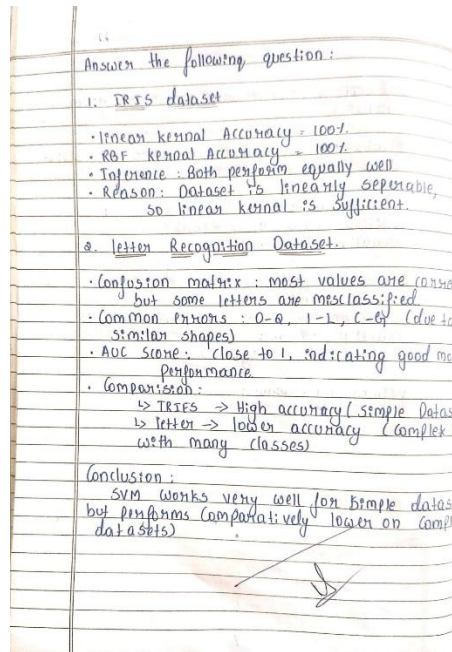
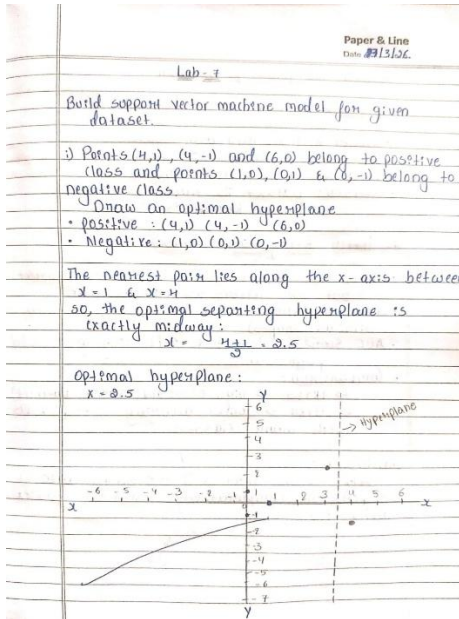
```



Program 7

Build Support vector machine model for a given dataset

Screenshot



Code:

```
# Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Load dataset
data = datasets.load_iris()
X = data.data
y = data.target

# Feature scaling (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset (fixed for same results every time)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:\n", cr)
```

... Accuracy: 0.9666666666666667

Confusion Matrix:

```
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot

Paper & Line
Date: 27/3/22

Lab - 08

Implement Random Forest ensemble method on a given dataset.

For sample S₁ shown, draw the decision Tree considering C₁PA as a root node

Sl. No	C ₁ PA	Interactiveness	G.S	Job offer
1	≥ 9	Yes	Good	Yes
2	< 9	No	Good	Yes
3	≥ 9	No	Avg	No
4	≥ 9	No	Avg	No
5	≥ 9	Yes	Good	Yes

Decision Tree:

Root C₁PA

```

graph TD
    Root["C1PA"] -- "< 9" --> Yes1["Yes"]
    Root -- "≥ 9" --> Interactiveness["Interactiveness"]
    Interactiveness -- "Yes" --> Yes2["Yes"]
    Interactiveness -- "No" --> PracticalKnowledge["Practical knowledge"]
    PracticalKnowledge -- "Good" --> Yes3["Yes"]
    PracticalKnowledge -- "Avg" --> No1["No"]
  
```

Sl. No	C ₁ PA	Interactiveness	G.S	Practical knowledge
1	< 9	No	Moderate	Good
2	≥ 9	No	Moderate	Avg
3	≥ 9	No	Moderate	Avg
4	≥ 9	Yes	Moderate	Good
5	≥ 9	Yes	Moderate	Good

Job offer

Yes

No

No

Yes

Yes

Decision Tree

```

graph TD
    Root["Interactiveness"] -- "Yes" --> Yes1["Yes"]
    Root -- "No" --> PracticalKnowledge["Practical knowledge"]
    PracticalKnowledge -- "Good" --> Yes2["Yes"]
    PracticalKnowledge -- "Average" --> No1["No"]
  
```

3. Random Forest - Iris dataset

- Best accuracy: 1.0 (100%)
- Best trees: 10

Confusion Matrix

	10	0	0
	0	9	0
	0	0	11

Inference:

- The model achieved perfect classification with no misclassifications.
- Increasing the number of trees did not improve accuracy since the dataset is simple.

Code:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
... Accuracy: 1.0
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot

3/4/26

Lab - 9

Implement Boosting ensemble method on a given dataset.

1) Considering AdaBoost Algorithm. For the following sample data, show the Decision stump calculation steps for the attribute (GPA).

GPA	Intern	P.k	C.S	Job profile
≥ 9	Yes	Good	Good	Yes
< 9	No	Good	Moderate	Yes
≥ 9	No	Avg	Moderate	No
< 9	No	Avg	Good	No
≥ 9	Yes	Good	Moderate	Yes
≥ 9	Yes	Good	Moderate	Yes

Step 1: Total Samples
Total = 6

Assume initial weights:
Each sample weight = 1/6

Step 2: Split on GPA

Case 1: GPA ≥ 9
Samples = 4 → Output: yes, No, yes, yes
Majority = yes
Misclassified = 1

Case 2: GPA < 9
Samples = 2 → Output: yes, No
Majority = Tie (Consider 1 misclassification)

Paper & Line
Date

Misclassified = 1

Step 3: Total Misclassification
Total misclassified = 2

Step 4: Error Calculation
 $Error = \frac{2}{6} = 0.33$

• Decision stump: Based on GPA
• Error = 0.33

Inference:
(GPA alone is not strong classifier since it misclassifies 2 samples)

2) AdaBoost - Income Dataset Result

• Best Accuracy Score: 0.8335 (~83.35%)
• Number of trees (in estimations): 80

Confusion Matrix:

	IN	FP
	FN	TP

(From the model predictions show high correct classification with fewer misclassifications)

Inference:

- Accuracy improves as the number of trees increases.
- Best performance is achieved at 80 trees, after which it stabilizes

Code:

```
# Import libraries
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

... Accuracy: 0.9444444444444444

Classification Report:

	precision	recall	f1-score	support
0	0.93	1.00	0.97	14
1	0.93	0.93	0.93	14
2	1.00	0.88	0.93	8
accuracy			0.94	36
macro avg	0.95	0.93	0.94	36
weighted avg	0.95	0.94	0.94	36

```
# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

... Accuracy: 0.9444444444444444

Classification Report:

	precision	recall	f1-score	support
0	0.93	1.00	0.97	14
1	0.93	0.93	0.93	14
2	1.00	0.88	0.93	8
accuracy			0.94	36
macro avg	0.95	0.93	0.94	36
weighted avg	0.95	0.94	0.94	36

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot

10/4/2020 Lab-10 (PCA)

1) Given the data in table, reduce the dimension from 4 to 1 using principle component analysis (PCA). Compute for 1st principle component.

Feature	Ex1	Ex2	Ex3	Ex4
x_1	4	8	13	7
x_2	11	4	5	14

Consider eigen values as follows:

$\lambda_1 = 30.3849$
 $\lambda_2 = 6.6151$

Consider eigen vectors as follows:

$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$ $e_2 = \begin{bmatrix} 0.8303 \\ 0.5574 \end{bmatrix}$

⇒ Step 1: Form Dataset

x	4	8	13	7
y	11	4	5	14

Step 2: Mean centering

Compute mean:

$\bar{x}_1 = (4+8+13+7)/4 = 8$
 $\bar{x}_2 = (11+4+5+14)/4 = 8.5$

centered data:

x'	-4	-2.5
	0	-4.5
	5	-3.5
	-1	5.5

Step 3: Use given eigen values & eigen vectors.

$\lambda_1 = 30.3849$, $\lambda_2 = 6.6151$
 Since λ_1 is largest we take:
 PCA uses eigenvector:
 $e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$

Step 4: Compute PCA

Formula = $Z = x' \cdot e_1$

Now compute for each example.

Ex 1:
 $(-4)(0.5574) + (-2.5)(-0.8303) = -2.2296 - 2.0757$
 $= -4.30535$

Example 2:
 $(0)(0.5574) + (-4.5)(-0.8303) = 0 + 3.73635$
 $= 3.73635$

Example 3:
 $(5)(0.5574) + (-3.5)(-0.8303) = 2.787 + 2.90605$
 $= 5.69305$

Example 4:
 $(-1)(0.5574) + (5.5)(-0.8303) = -0.5574 - 4.56665$
 $= -5.12405$

$Z =$	-4.305
	3.736
	5.693
	-5.124

2) Accuracy Before and After applying PCA

⇒ Accuracy before applying PCA

- SVM: 0.8750
- Logistic Regression: 0.8533
- Random Forest: 0.8804

⇒ Accuracy After applying PCA

- SVM: 0.8804
- Logistic Regression: 0.8533
- Random Forest: 0.8641

Result:

After Applying PCA:

- SVM accuracy slightly increased
- Logistic Regression remained same
- Random Forest accuracy decrease

This shows that

- can sometimes improve or maintain accuracy

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
▶ data = pd.read_csv("/content/data.csv")
print(data.head())
```

```
X = data.iloc[:, :].values # or choose specific columns
```

```
k = 3 # you can change this
```

```
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)
```

```
labels = kmeans.labels_
print(labels)
```

```
▶ centroids = kmeans.cluster_centers_
print(centroids)
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red')
plt.title("K-Means Clustering")
plt.show()
```


Program 11

Implement Dimensionality reduction using Principle Component Analysis (PCA) method.

Screenshot

Paper & Line
Date: 10/11/20

lab-11

Build k-means algo to cluster a set of data stored in csv file.

1) For the given dataset compute two clusters using k-means algorithm for clustering where initial cluster centers are (1,0, 1.0) and (5,0, 7.0). Execute for two iterations.

Record Number	A	B
R ₁	1.0	1.0
R ₂	1.5	2.0
R ₃	3.0	4.0
R ₄	5.0	7.0
R ₅	3.5	5.0
R ₆	4.5	5.0
R ₇	3.5	4.5

Initial centroids
 $C_1 = (1, 1)$
 $C_2 = (5, 7)$
we use Euclidean Distance
 $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

After 2 iterations

Cluster 1:
R₁, R₂
Centroid: (1.25, 1.5)

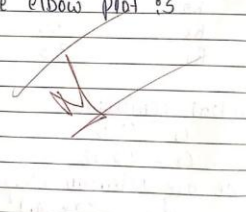
Cluster 2:
R₃, R₄, R₅, R₆, R₇
Centroid: (3.9, 5.1)

2) Observation:

- Elbow plot
- The elbow plot method was used to determine the optimal number of clusters.
- A graph was plotted between:
 - Number of clusters (k) on x-axis
 - wcss (within cluster sum of squares) on y-axis
- As k increases, wcss decreases
- the graph shows a sharp bend (elbow point) at k=3

Result:

- The optimal number of clusters obtained from the elbow plot is k=3.



Code:

PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

```
# Load dataset
data = pd.read_csv("data.csv")
print(data.head())
```

```
...      x  y
0  30  49
1  31  27
2  31  46
3   1  65
4  81  76
```

```
# Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

```
# Apply PCA
pca = PCA(n_components=2) # Reduce to 2 components
pca_result = pca.fit_transform(scaled_data)
```

```
# Explained variance ratio
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")
print(f"Total Variance Explained by 2 Components: {np.sum(pca.explained_variance_ratio_)}")
```

```
... Explained Variance Ratio: [0.56506606 0.43493394]
Total Variance Explained by 2 Components: 1.0
```

```
# Scatter plot of the 2D PCA result
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c='blue', edgecolors='black')
plt.title('PCA Result (2 Components)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```

